

DecodMETAR: validador e decodificador de boletins METAR/SPECI com Expressões Regulares

Relatório técnico, Trabalho do 1º Bimestre

Disciplina: Linguagens Formais e Autômatos (CESUPA)

Equipe: Gabriel Albuquerque Alencar, Davi Maciel Corrêa, José Valdez, Alberto Acosta

Repositório: <https://github.com/Gaalbu/decodmetar>

Data: 30/09/2026

1. Introdução

Expressões Regulares (ERs) descrevem linguagens regulares e são usadas no dia a dia da computação para validar e extrair informação de textos com formato fixo. Este trabalho aplica ERs a um problema real: a leitura de boletins meteorológicos aeronáuticos.

O objetivo foi construir uma aplicação que resolva esse problema com pelo menos cinco ERs relevantes e mostrar, para cada uma, que a **ER formal**, o **padrão no código**, os **testes** e o **AFNe** representam a mesma linguagem, como exige o guia da disciplina. O projeto usa oito ERs e verifica essa equivalência automaticamente.

2. Problema e solução

2.1 O problema

O METAR é o boletim que os aeroportos emitem de hora em hora com as condições do tempo; o SPECI é a versão especial, emitida quando o tempo muda muito. O formato é definido pela ICAO (Anexo 3) e, no Brasil, pelo DECEA. Um exemplo:

```
METAR SBBE 241200Z 09010KT 9999 -RA FEW020 SCT100 30/24 Q1011
```

Para quem não é da área, isso é ilegível. E, como é digitado ou transmitido em texto puro, um erro (dia 32, vento de 370°, pressão 1100 hPa) pode passar despercebido.

2.2 A solução

O DecodMETAR é um programa de linha de comando em Python:

Entrada	Um boletim digitado (menu ou argumento) ou um arquivo <code>.txt</code> com um boletim por linha.
Processamento	Separa o boletim em grupos, valida o cabeçalho com a ER-01, encaixa cada grupo seguinte na próxima posição esperada (ER-02 a ER-08), extrai os valores e traduz.
Saída	O boletim traduzido grupo a grupo; os grupos inválidos com o motivo; avisos semânticos; no modo arquivo, um resumo.

A mesma linha acima vira (saída abreviada; a completa está no README):

```
METAR SBBE  ER-01  METAR (observação regular) do aeródromo SBBE, Belém/Val-de-Cans (PA)
241200Z      ER-02  Observação do dia 24 às 12:00 UTC (09:00 no horário de Brasília)
09010KT      ER-03  Vento de 090° (vindo do leste) a 10 nós (19 km/h)
9999         ER-04  Visibilidade de 10 km ou mais
-RA          ER-05  Tempo presente: chuva (intensidade fraca)
FEW020       ER-06  Poucas nuvens (1 a 2 oitavos do céu) a 2.000 pés (610 m)
SCT100       ER-06  Nuvens esparsas (3 a 4 oitavos) a 10.000 pés (3.048 m)
30/24        ER-07  Temperatura de 30 °C e ponto de orvalho de 24 °C (umidade de cerca de 70%)
Q1011        ER-08  Pressão ajustada ao nível do mar (QNH) de 1.011 hPa
```

O programa também tem um modo de teste de ERs: o usuário escolhe uma ER, digita qualquer cadeia e vê o resultado do `re.fullmatch`, o resultado do AFNε e o traço do AFNε símbolo a símbolo. É o modo usado na demonstração.

3. Fundamentação teórica

3.1 Expressões Regulares

Sobre um alfabeto finito Σ , as ERs são definidas a partir de $\emptyset$, ϵ e dos símbolos de Σ , combinados por **união** ($r \mid s$), **concatenação** (rs) e **fecho de Kleene** (r^*). Os demais operadores usados são abreviações:

Operador	Equivalência formal
r^+ (fecho positivo)	$r r^*$
$r?$ (opcionalidade)	$(r \mid \epsilon)$
$[abc]$ (classe finita)	$(a \mid b \mid c)$
$[\emptyset-9]$ (intervalo)	$(\emptyset \mid 1 \mid \dots \mid 9)$
$r\{m\}$ (repetição exata)	m cópias de r concatenadas
$r\{m,n\}$ (repetição limitada)	$r^m \mid r^{m+1} \mid \dots \mid r^n$

A precedência é: repetição, depois concatenação, depois união.

3.2 AFNε

Um autômato finito não determinístico com movimentos vazios é uma quintupla $(Q, \Sigma, \delta, q_0, F)$, em que $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow P(Q)$. Os movimentos ϵ permitem mudar de estado sem consumir símbolo. O **fecho-ε** de um conjunto de estados é o conjunto de todos os estados

alcançáveis a partir dele só com movimentos ϵ . Uma cadeia é aceita se, partindo do fecho- ϵ de q_0 e alternando "ler um símbolo" e "tomar o fecho- ϵ ", o conjunto final contém algum estado de F .

3.3 Equivalência

Pelo teorema de Kleene, toda ER tem um autômato finito que reconhece a mesma linguagem, e vice-versa. A construção de Thompson dá o caminho de ER para AFN ϵ : a união vira uma bifurcação com ϵ , a concatenação liga autômatos em sequência e o fecho vira um laço com ϵ . Os AFN ϵ deste trabalho seguem essa ideia de forma compacta (sem os estados redundantes da construção "pura"), para caberem num slide.

3.4 Da notação formal para o Python

O motor `re` do Python tem sintaxe própria. As diferenças que importam aqui:

- **Correspondência completa:** usamos `re.fullmatch`, que exige que a cadeia inteira pertença à linguagem. Assim os padrões não precisam das âncoras `^` e `$`, que não são símbolos do alfabeto.
- **Escape:** `+` é metacaractere; como símbolo literal (intensidade forte na ER-05) é escrito `\+`.
- **Classes explícitas:** não usamos `\d`, `\w`, `\s` nem `.`, que dependem do motor e do Unicode. Todas as classes são explícitas (`[0-9]`, `[A-Z]`), então o alfabeto de cada ER fica declarado.
- **Recursos proibidos:** nenhuma ER usa retroreferências, lookahead, recursão ou condicionais.
- **Grupos de captura:** os parênteses `()` do código servem só para agrupar. Os valores são extraídos fatiando o grupo já validado, e não com grupos nomeados, para o padrão ficar idêntico ao da ficha.

4. Metodologia e arquitetura

4.1 Organização do código

<code>main.py</code>	interface: menu interativo e linha de comando
<code>decodmetar/padroes.py</code>	as 8 ERs (única fonte dos padrões)
<code>decodmetar/decodificador.py</code>	tokenização, ordem dos grupos, diagnóstico, tradução
<code>decodmetar/mensagens.py</code>	tabelas de tradução e textos
<code>decodmetar/afne/automato.py</code>	classe AFN ϵ : fecho- ϵ , mover, aceitação, traço
<code>decodmetar/afne/definicoes.py</code>	os 8 AFN ϵ como dados (estados, transições, ϵ)
<code>decodmetar/afne/diagramas.py</code>	gera os diagramas Mermaid e as tabelas de transição
<code>tests/</code>	casos de teste e 71 testes automáticos

4.2 Fluxo da decodificação

1. **Limpeza:** remove espaços das pontas e o `=` final (os boletins reais terminam com `=`). Entrada vazia gera a mensagem "Entrada vazia: digite um boletim METAR ou SPECI."
2. **Tokenização:** `str.split()`. Não usamos uma ER para isso, para não criar uma nona ER não documentada.
3. **Cabeçalho:** junta os 2 primeiros grupos (ou 3, se o segundo for `COR`) e testa na ER-01. Se falhar, o boletim é rejeitado com o motivo.

4. **Corpo:** os grupos seguintes são encaixados em ordem: data/hora → vento → visibilidade → tempo presente (0 a 3) → nuvens (0 a 4) → temperatura → pressão. Para cada grupo, o programa procura a primeira posição, a partir da atual, cuja ER aceite o grupo.
5. **Grupo que não casa:** o programa descobre se ele está fora de ordem, repetido ou depois de `CAVOK`. Senão, "adivinha" pela aparência que grupo ele tentou ser (termina em `KT` → vento; tem `/` → temperatura...) e explica o erro ("direção 370° acima de 360°"). As funções de diagnóstico usam só métodos de string, sem ERs extras.
6. **Grupos ignorados:** `AUTO` e `NOSIG` são pulados; depois de `RMK`, `BECMG` ou `TEMPO`, o resto da linha é ignorado.
7. **Avisos:** checagens que dependem de comparar números, e que uma ER não faz, viram avisos: orvalho maior que a temperatura, rajada que não é maior que a média, direção 000 com vento.

A ordem dos grupos também é uma linguagem regular, sobre o alfabeto dos "tipos de grupo" (`ER02 ER03 ER04 ER05{0,3} ER06{0,4} ER07 ER08`). O decodificador a implementa como uma máquina de posições, que funciona como um autômato sobre esses tipos.

4.3 Os AFNε como dados

Cada AFNε é declarado como um dicionário `{estado: [(rótulo, destino), ...]}`. O rótulo é um símbolo, uma classe finita no formato do guia (`[0-9]`, `[BDINSW]`) ou `ε`. O simulador expande cada classe para o conjunto de símbolos que ela abrevia.

Os diagramas (`docs/afne/ERxx.md` e `.svg`) são **gerados** a partir dessas mesmas definições. Assim o desenho do slide é exatamente o autômato testado; um teste falha se alguém mudar o AFNε e esquecer de gerar os diagramas.

4.4 Ferramentas

Python ≥ 3.10, somente biblioteca padrão (`re`, `unittest`, `argparse`, `dataclasses`, `pathlib`, `random`, `math`); Git e GitHub; Mermaid para os diagramas (renderizado pelo GitHub e exportado para SVG).

5. As Expressões Regulares

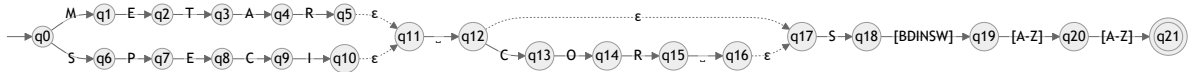
As fichas completas, no formato do guia (identificação, alfabeto, linguagem, ER formal, sintaxe, equivalência dos operadores, AFNε, ≥ 6 aceitas, ≥ 6 rejeitadas com motivo, casos-limite, resultado e limitações), estão em `docs/EXPRESSOES_REGULARES.md`. Aqui vai o resumo de cada uma, com o AFNε.

Notação: `D = (0 | ... | 9)`, `L = (A | ... | Z)`, `_` = espaço. Nos diagramas, setas tracejadas são movimentos ε, o círculo duplo é o estado final e a seta sem origem aponta o estado inicial.

ER-01: Identificação do boletim

- **Σ:** `L U {_}`
- **L:** `METAR` ou `SPECI`, espaço, `COR_` opcional e indicativo ICAO brasileiro (`S + B/D/I/N/S/W + 2` letras).

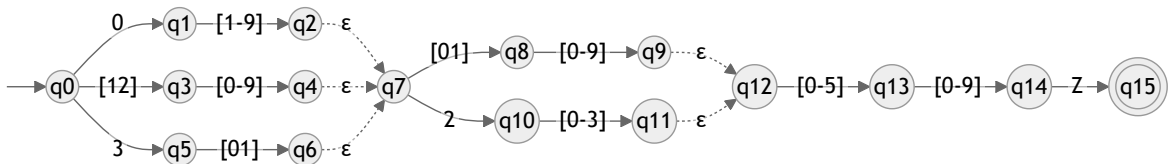
- **ER formal:** (METAR | SPECI) (COR) S (B | D | I | N | S | W) L L
- **Código:** r"(METAR|SPECI) (COR)?S[BDINSW][A-Z]{2}"
- **AFNε:** 22 estados, inicial q0, final q21, 4 movimentos ε.



ER-02: Data e hora

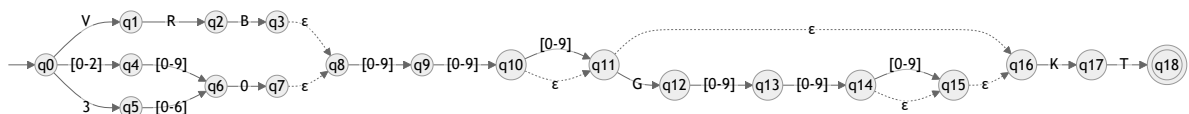
- **Σ:** D ∪ {Z}
- **L:** dia 01–31, hora 00–23, minuto 00–59 e Z.
- **ER formal:** (0 (1 | ... | 9) | (1 | 2) D | 3 (0 | 1)) ((0 | 1) D | 2 (0 | 1 | 2 | 3)) (0 | 1 | 2 | 3 | 4 | 5) D Z
- **Código:** r"(0[1-9]|1[12][0-9]|3[01])([01][0-9]|2[0-3])[0-5][0-9]Z"
- **AFNε:** 16 estados, inicial q0, final q15, 5 movimentos ε.

Ponto central: ER não compara números, então cada intervalo ("01 a 31") vira uma união de padrões de algoritmos.



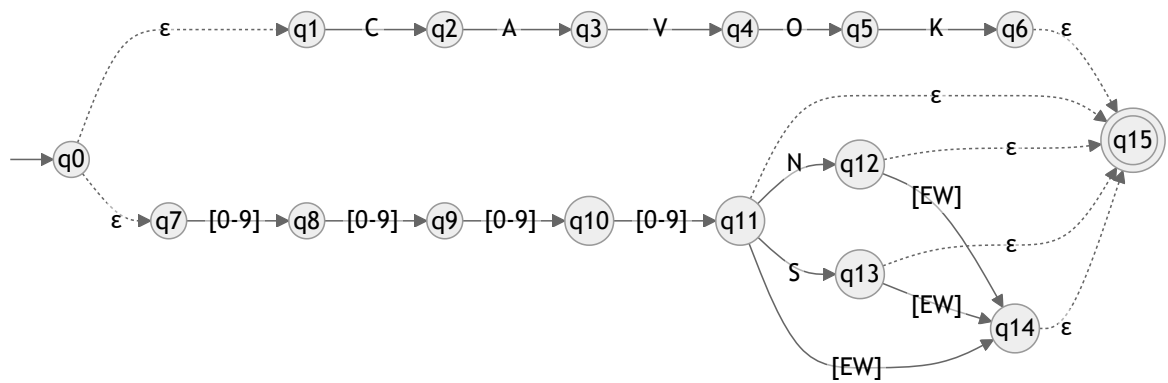
ER-03: Vento

- **Σ:** D ∪ {V, R, B, G, K, T}
- **L:** VRB ou direção 000–360 em dezenas, velocidade com 2 ou 3 algoritmos, rajada opcional e KT.
- **ER formal:** (VRB | ((0 | 1 | 2) D | 3 (0 | 1 | 2 | 3 | 4 | 5 | 6)) 0) D D (D | ε) (G D D (D | ε) | ε) K T
- **Código:** r"(VRB|([0-2][0-9]|3[0-6])0)[0-9]{2,3}(G[0-9]{2,3})?KT"
- **AFNε:** 19 estados, inicial q0, final q18, 6 movimentos ε.



ER-04: Visibilidade

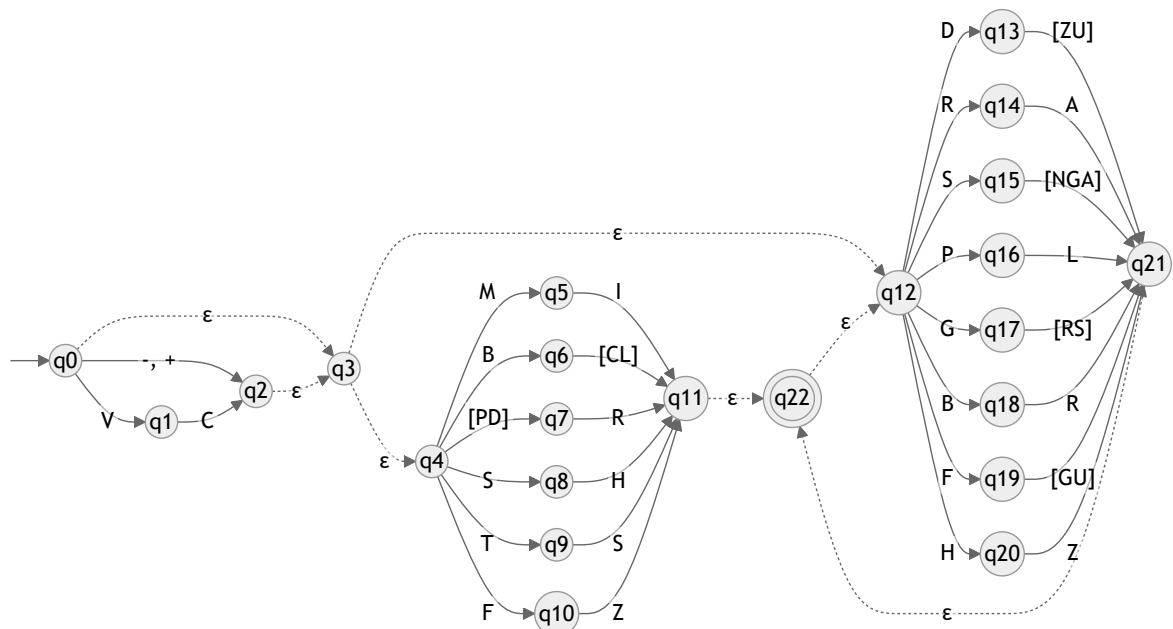
- **Σ:** D ∪ {C, A, V, O, K, N, E, S, W}
- **L:** CAVOK, ou 4 algoritmos com direção opcional (N, NE, E, SE, S, SW, W, NW).
- **ER formal:** CAVOK | D D D D (N | NE | E | SE | S | SW | W | NW | ε)
- **Código:** r"CAVOK|[0-9]{4}(N|NE|E|SE|S|SW|W|NW)?"
- **AFNε:** 16 estados, inicial q0, final q15, 7 movimentos ε.



ER-05: Tempo presente

- **Σ :** { -, +, A, B, C, D, F, G, H, I, L, M, N, P, R, S, T, U, V, Z }
- **Definições:** Desc = (MI | BC | PR | DR | BL | SH | TS | FZ), Fen = (DZ | RA | SN | SG | PL | GR | GS | BR | FG | FU | HZ | DU | SA)
- **L:** intensidade/proximidade opcional (-, +, VC), seguida de um descritor com zero ou mais fenômenos, ou de um ou mais fenômenos.
- **ER formal:** (' - ' | ' + ' | VC | ε) (Desc Fen* | Fen Fen*)
- **Código:** `r"(-|\+|VC)?((MI|BC|PR|DR|BL|SH|TS|FZ)(DZ|RA|SN|SG|PL|GR|GS|BR|FG|FU|HZ|DU|SA)*|(DZ|RA|SN|SG|PL|GR|GS|BR|FG|FU|HZ|DU|SA)+)"`
- **AFNε:** 23 estados, inicial q0, final q22, 7 movimentos ε.

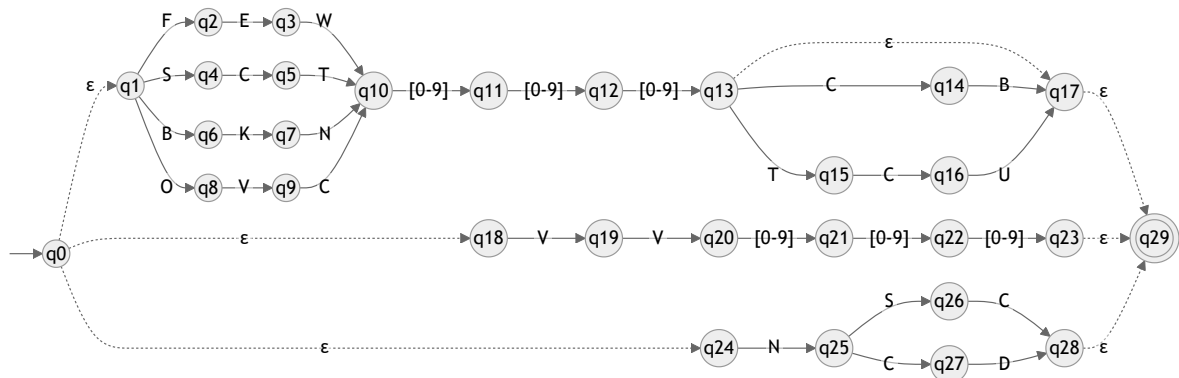
É a única ER com fecho (* e +) e a única com linguagem infinita. O AFNε usa a distributividade `Desc Fen* | Fen Fen* = (Desc | Fen) Fen*`: lê um descritor ou um fenômeno e depois usa um único laço ε (q22 → q12) para os fenômenos seguintes.



ER-06: Nuvens

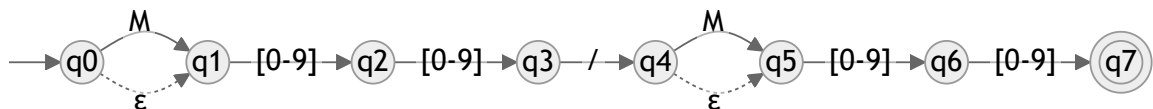
- **Σ :** D U { B, C, D, E, F, K, N, O, S, T, U, V, W }
- **L:** quantidade + altura (3 algarismos) + tipo opcional; ou vv + 3 algarismos; ou NSC; ou NCD.

- **ER formal:** (FEW | SCT | BKN | OVC) D D D (CB | TCU | ε) | VV D D D | NSC | NCD
- **Código:** r"(FEW|SCT|BKN|OVC)[0-9]{3}(CB|TCU)?|VV[0-9]{3}|NSC|NCD"
- **AFNε:** 30 estados, inicial q0, final q29, 7 movimentos ε.



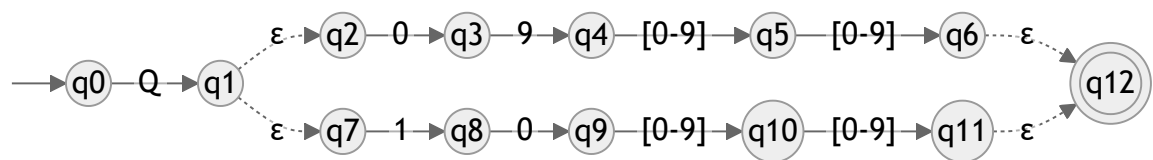
ER-07: Temperatura e ponto de orvalho

- **Σ:** D ∪ {M, /}
- **L:** 2 algarismos com M opcional (negativo), /, idem.
- **ER formal:** (M | ε) D D / (M | ε) D D
- **Código:** r"M?[0-9]{2}/M?[0-9]{2}"
- **AFNε:** 8 estados, inicial q0, final q7, 2 movimentos ε.



ER-08: Pressão QNH

- **Σ:** D ∪ {Q}
- **L:** Q + valor de 0900 a 1099 hPa.
- **ER formal:** Q (0 9 D D | 1 0 D D)
- **Código:** r"Q(09[0-9]{2}|10[0-9]{2})"
- **AFNε:** 13 estados, inicial q0, final q12, 4 movimentos ε.



6. Testes e análise dos resultados

6.1 Como foi testado

Todos os testes usam só o `unittest` e rodam com `python -m unittest discover -s tests -v`. São 71 testes em 7 arquivos:

Arquivo	O que verifica
test_padroes.py	Cada ER aceita as cadeias aceitas e rejeita as rejeitadas; cada ER tem ≥ 6 de cada e ≥ 1 caso-limite.
test_afne.py	O AFNε dá o mesmo resultado nos casos fixos e concorda com a regex em cadeias geradas.
test_automato.py	O simulador (fecho-ε, aceitação, traço) num AFNε pequeno feito à mão.
test_diagramas.py	Os diagramas têm todas as transições e estão atualizados.
test_decodificador.py	Boletins válidos, todos os tipos de erro, avisos e leitura de arquivos.
test_main.py	Linha de comando e menu, incluindo entradas inválidas.
test_documentacao.py	Cada ficha mostra o mesmo padrão, ER formal, cadeias e traços que o código produz.

O teste de equivalência regex × AFNε gera, para cada ER, com semente fixa (2026):

1. 2000 cadeias aleatórias sobre o alfabeto da ER mais símbolos estranhos (x , espaço, - , /);
2. todas as cadeias a uma edição (trocar, remover ou inserir um símbolo) das cadeias aceitas;
3. cadeias geradas andando ao acaso pelo próprio AFNε até um estado final, e as mutações de 200 delas.

O item 3 é o que pega um AFNε que aceita **mais** do que a regex; o item 2 pega erros na fronteira da linguagem.

6.2 Resultados

Resultado real da execução (docs/resultado_testes.txt , gerado por `python -m tests.relatorio`):

ER	Aceitas	Rejeitadas	Limites	Regex ok	AFNε ok
ER-01	7	9	4	16/16	16/16
ER-02	6	9	5	15/15	15/15
ER-03	7	9	5	16/16	16/16
ER-04	7	8	4	15/15	15/15
ER-05	10	10	5	20/20	20/20
ER-06	8	9	4	17/17	17/17
ER-07	6	9	4	15/15	15/15
ER-08	6	9	5	15/15	15/15

ER	Cadeias comparadas	Aceitas entre elas	Divergências regex × AFNε
ER-01	146.913	8.898	0
ER-02	46.610	6.903	0
ER-03	80.776	14.222	0
ER-04	52.795	8.910	0
ER-05	85.864	2.870	0
ER-06	73.962	4.272	0
ER-07	40.846	8.749	0
ER-08	24.046	200	0

Os 71 testes passaram.

6.3 Análise

- **Casos fixos:** as 129 cadeias das fichas deram o resultado esperado tanto na regex quanto no AFNε.
- **Equivalência:** em cerca de 552 mil cadeias distintas, regex e AFNε não discordaram nenhuma vez. Na ER-08, cuja linguagem é finita e tem exatamente 200 cadeias (`Q0900` a `Q1099`), os passeios pelo AFNε encontraram as 200: a comparação cobriu a linguagem inteira.
- **O teste realmente pega erros?** Para confirmar, plantamos 14 defeitos nos AFNε (um intervalo trocado, um ε removido, um laço a mais...) e rodamos de novo. Todos foram detectados. Exemplos: trocar `[0-3]` por `[0-4]` na hora da ER-02 fez o teste apontar `012402Z`; remover o laço `q22 → q12` da ER-05 fez apontar `+FZDZ`; um ε extra `q2 → q0` na ER-05 fez apontar `+++DR`. A lista completa está na seção final de `docs/EXPRESSOES_REGULARES.md` ("Como garantimos que a ER e o AFNε reconhecem a mesma linguagem").
- **Decodificador:** todos os boletins de `dados/metar_validos.txt` (10) são aceitos e todos os de `dados/metar_com_erros.txt` (12, um erro diferente em cada) são rejeitados com a mensagem certa.
- **Limite do método:** testes não são prova. A prova formal seria converter os dois lados para AFD mínimo e comparar. Consideramos a evidência suficiente porque ela combina casos fixos, fronteira, passeios pelo autômato e a detecção de todos os defeitos plantados.

7. Limitações e possíveis melhorias

Limitações das ERs (forma × sentido). As ERs validam o formato, não o significado:

- ER-01 aceita aeródromos inexistentes (`METAR SIZZ`) e não inclui o bloco `SJ`.
- ER-02 aceita o dia 31 em qualquer mês; o METAR nem informa o mês.
- ER-03 aceita rajada menor que a média (`09020G10KT`); o decodificador avisa.
- ER-05 aceita combinações sem sentido (`-TS`, `RARARA`) e não inclui códigos raros (`SQ`, `FC`, `UP`, `RE...`).
- ER-06 não aceita `SKC/CLR` (não usados no Brasil) nem `///` de estações automáticas.

- ER-07 aceita orvalho maior que a temperatura; o decodificador avisa.
- ER-08 rejeita pressões fora de 900–1099 hPa.

Limitações do programa. Grupos menos comuns (variação de direção `350V040`, RVR, tendências `BECMG` / `TEMPO`, remarks) são ignorados, não decodificados. O TAF (previsão) está fora do escopo.

Possíveis melhorias:

- decodificar os grupos ignorados (cada um seria uma nova ER com ficha e AFNε);
- validar a ordem crescente das camadas de nuvens;
- aceitar a data com o mês informado pelo usuário, para checar o calendário;
- buscar METAR reais pela internet (API da REDEMET) em vez de digitar;
- interface web;
- prova formal da equivalência via minimização de AFD.

8. Uso de Inteligência Artificial

A equipe usou IA (Claude, da Anthropic) para:

- elaborar o plano de execução e sugerir as ERs e os casos de teste;
- escrever parte do código (ERs, AFNε, decodificador, interface) e dos testes;
- redigir parte da documentação (README, fichas e este relatório);
- extrair o texto do guia da disciplina para conferir o formato das fichas.

A equipe revisou todo o conteúdo e é responsável por entendê-lo, explicá-lo e modificá-lo. Os resultados de teste deste relatório foram copiados da execução real, não escritos à mão.

9. Contribuições

Integrante	Responsabilidade
Davi Maciel Corrêa	ER-01 e ER-02; decodificador e mensagens
Gabriel Albuquerque Alencar	ER-03 e ER-04; simulador de AFNε, diagramas e verificação de equivalência
José Valdez	ER-05 e ER-06; interface (menu e linha de comando), casos de teste e dados de exemplo
Alberto Acosta	ER-07 e ER-08; fichas, relatório, slides e contribuições

Registro detalhado em `CONTRIBUICOES.md` e no histórico do Git.

10. Referências

- Guia de Sintaxe para Apresentação das Expressões Regulares. Material da disciplina Linguagens Formais e Autômatos, CESUPA, 2026.
- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. Rio de Janeiro: Elsevier.

- ICAO. *Annex 3 to the Convention on International Civil Aviation: Meteorological Service for International Air Navigation*.
- DECEA. *Manual de Códigos Meteorológicos* (MCA 105-10).
- WMO. *Guide to Instruments and Methods of Observation* (WMO-No. 8): fórmula de Magnus para a umidade relativa.
- PYTHON SOFTWARE FOUNDATION. *re: Regular expression operations*.
<https://docs.python.org/3/library/re.html>
- Mermaid. <https://mermaid.js.org>